

Secret Leak Detection in Software Issue Reports using LLMs: A Comprehensive Evaluation

Sadif Ahmed^{1,*} Md Nafiu Rahman^{1,*} Zahin Wahab^{2,*} Gias Uddin³ Rifat Shahriyar¹

¹Bangladesh University of Engineering and Technology

²The University of British Columbia

³York University

*Equal contribution

MSR 2026 | Rio de Janeiro, Brazil

The Problem: Secrets in Software

What Are Secrets?

API keys, OAuth tokens, RSA keys, passwords. These are the digital credentials that protect access to sensitive systems.

- **10 Million** hardcoded secrets found in 2022, a **67% jump** from 2021 [3].
- Nearly **half of all cyberattacks** in early 2023 used stolen credentials [10].
- A **1212x surge** in leaked OpenAI API keys reported in 2024 [4].

The Overlooked Risk

Most prior work focuses on **source code**. But secrets also leak in **software issue reports**, which are unstructured, noisy, and largely unmonitored.

The Overlooked Attack Surface: Issue Reports

Why Issue Reports Are Risky

Developers often paste logs, stack traces, shell commands, and example credentials when describing bugs or asking for help.

A real leak in an issue report:

```
curl https://api.stripe.com/v1/charges \  
-u sk_test_4eC39HqLyjWDarjtT1zdp7dc
```

AWS credentials in a debug log:

```
aws_access_key_id=AKIAIOSFODNN7EXAMPLE
```

vs. A harmless placeholder:

```
apiKey=YOUR_API_KEY  
token=sk_test_4eC39HqLyjWxxxxxxx
```

Real secrets and placeholders sit side by side in messy, unstructured text. This makes detection much harder than in source code.

Example: Secret Leak in a GitHub Issue Report

I'm trying to call the API, but it keeps refusing authentication.

```
import requests

API_KEY = "12345-abcde-67890-fghij"
response = requests.get("https://api.example.com/data",
    headers={"Authorization": API_KEY}
)
```

Figure: A real API key mistakenly shared in a public issue report (masked for security).

Why Traditional Tools Fall Short

Current Tools Are Not Enough

Tools like TruffleHog [11], Gitleaks [5], and ggshield [2] rely on regex and entropy analysis.

- **Too many false positives:** Regex flags things that *look like* secrets, such as placeholders, dummy keys, and test values.
- **No understanding of context:** Cannot tell a real API key apart from `apiKey=YOUR_KEY_HERE`.
- **Built for source code:** Not tested or designed for the noisy, unstructured text of issue reports.
- **No tools exist for issue reports:** Our survey found that **zero** developers use automated tools for issue reports.

The Result

Too many false alarms lead to **alert fatigue**, so developers start ignoring warnings and miss real threats.

Related Work & The Research Gap

Prior Work Focuses on Source Code

- **Large-scale analysis** (Meli et al. [6]): Scanned billions of files with regex and entropy, but limited in pattern coverage.
- **Program analysis** (Sinha et al. [9]): Precise slicing for specific APIs, but not usable on unstructured text.
- **ML & DL approaches** (Saha et al. [8], Feng et al. [1]): Depend on code-specific features or target only narrow secret types.
- **LLM-based detection** (Rahman et al. [7]): Uses context to cut down false positives, but only works on source code.

Related Work & The Research Gap

Prior Work Focuses on Source Code

- **Large-scale analysis** (Meli et al. [6]): Scanned billions of files with regex and entropy, but limited in pattern coverage.
- **Program analysis** (Sinha et al. [9]): Precise slicing for specific APIs, but not usable on unstructured text.
- **ML & DL approaches** (Saha et al. [8], Feng et al. [1]): Depend on code-specific features or target only narrow secret types.
- **LLM-based detection** (Rahman et al. [7]): Uses context to cut down false positives, but only works on source code.

The Gap We Address

Secret detection in **issue reports**, where natural language, crash logs, and code snippets all mix together, is still largely unexplored.

Research Questions

- **RQ1:** What do developers think about the likelihood, severity, and causes of secret leaks in GitHub issue reports?

Research Questions

- **RQ1:** What do developers think about the likelihood, severity, and causes of secret leaks in GitHub issue reports?
- **RQ2:** How well do proprietary LLMs (GPT-4o, Gemini-2.0-flash) detect secrets in issue reports?

Research Questions

- **RQ1:** What do developers think about the likelihood, severity, and causes of secret leaks in GitHub issue reports?
- **RQ2:** How well do proprietary LLMs (GPT-4o, Gemini-2.0-flash) detect secrets in issue reports?
- **RQ3:** How well do smaller, open-source models detect secrets in issue reports?
 - **RQ3.1:** Smaller BERT-like models (CodeBERT, RoBERTa, XLNet, ...)
 - **RQ3.2:** Fine-tuned LLMs (LLaMA, Qwen, Mistral, DeepSeek, Gemma)

Our Contributions

What We Bring to the Table

- ① **First large-scale benchmark** for secret detection in issue reports: **54,148** labeled instances from **27,121** repos.

What We Bring to the Table

- ① **First large-scale benchmark** for secret detection in issue reports: **54,148** labeled instances from **27,121** repos.
- ② **Hybrid detection pipeline**: Regex extraction combined with LLM-based contextual classification.

Our Contributions

What We Bring to the Table

- ① **First large-scale benchmark** for secret detection in issue reports: **54,148** labeled instances from **27,121** repos.
- ② **Hybrid detection pipeline**: Regex extraction combined with LLM-based contextual classification.
- ③ **Broad evaluation** of proprietary (GPT-4o, Gemini) and open-source models (12 BERT-like + 5 LLMs).

Our Contributions

What We Bring to the Table

- ➊ **First large-scale benchmark** for secret detection in issue reports: **54,148** labeled instances from **27,121** repos.
- ➋ **Hybrid detection pipeline**: Regex extraction combined with LLM-based contextual classification.
- ➌ **Broad evaluation** of proprietary (GPT-4o, Gemini) and open-source models (12 BERT-like + 5 LLMs).
- ➍ **Real-world validation** on **178 unseen repositories**, reaching **81.6% F1-score**.

RQ1: What Developers Think (50 Participants)

RQ1: How do developers perceive the likelihood, severity, and causes of secret leaks in GitHub issue reports?

How Likely Are Leaks?

- **76%** rated likelihood 3 or higher out of 5
- 12% rated it at the maximum (5)

How Serious Are They?

- **68%** said highly severe
- 28% said moderately severe
- 4% said low severity

Why Do Secrets Leak?

Factor	%
Lack of awareness	44%
Urgency / time pressure	24%
Accidental oversight	12%
Team sharing	10%

How do they find leaks? Manual inspection (72%), noticed by others (28%), **automated tools: 0%**.

How We Built the Dataset

Building the Benchmark

- Collected issue reports from **27,121 public GitHub repos** using the REST API with targeted keywords (key, token, secret, ...).
- Used **761 regex patterns** to pull out candidate secrets from issue bodies.
- Filtered out harmless patterns (commit hashes, UUIDs) with 22 extra regexes.
- Every candidate was **manually labeled** as *Secret* or *Non-sensitive*.

Table: Final Dataset Statistics

Metric	Value
Total labeled instances	54,148
True secrets	5,881
Non-secrets	48,297
Source repositories	27,121
Inter-rater κ	0.9545

Secret Leaks Over Time

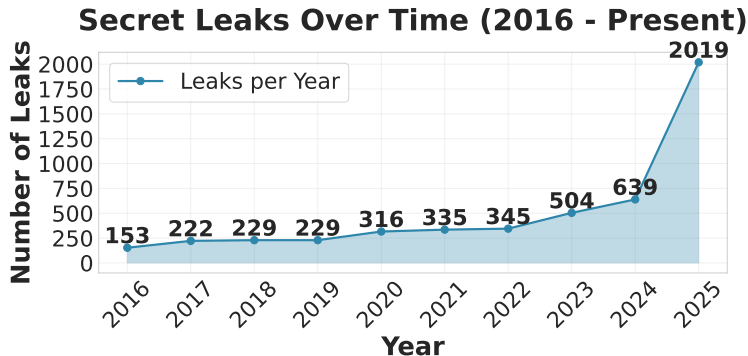


Figure: How secret leaks in issue reports have grown over the years (2016 to 2025). The sharp spike in 2025 reflects growing API and cloud usage.

How the Detection Pipeline Works

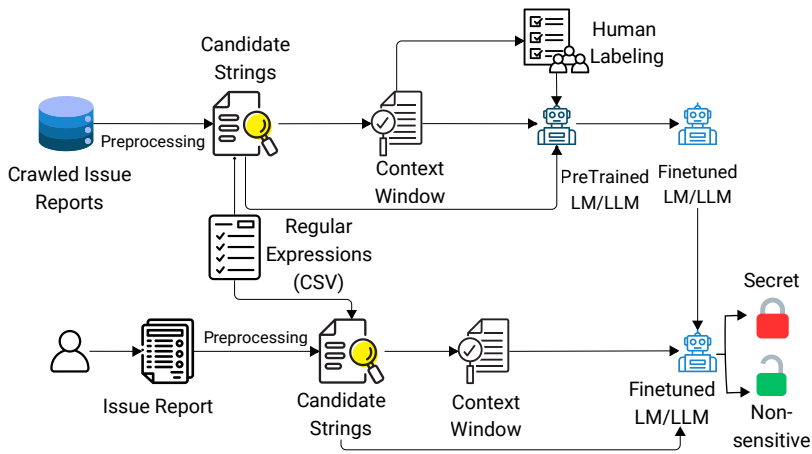


Figure: Our hybrid workflow: first extract candidates with regex, then classify them using an LLM.

Context Extraction

How We Extract Context

- We take a **200-character window** around each candidate string.
- This gives enough context for classification without reading the entire issue.

What Each Data Point Looks Like:

- 1 Candidate String (regex match)
- 2 Issue Context (200-char window)
- 3 Label: *Secret* or *Non-sensitive*

Classification Prompt

Task

Classify the candidate string as **Secret** or **Non-sensitive** based on its role in the issue report.

Secret if it is:

- An API key or token (e.g., `sk_test_ABC123`)
- A private/SSH/crypto key
- A password or database connection string with credentials
- An auth token (e.g., `Bearer <token>`)

Non-sensitive if it is:

- A public key or non-sensitive config value
- Clearly a dummy/test value (e.g., `test_key`, `EXAMPLE_KEY`)
- A placeholder or redacted string (e.g., `[REDACTED]`, `****`)
- Obfuscated or masked (e.g., `abc...xyz`)

Output

Return only the label: **Secret** or **Non-sensitive**.

RQ2: Proprietary LLMs (GPT-4o & Gemini)

RQ2: How well do proprietary LLMs (GPT-4o, Gemini-2.0-flash) detect secrets in issue reports?

Decent Results Without Fine-Tuning

Proprietary models show good recall but have trouble with precision.

Table: Proprietary LLM Performance

Setting	Model	Precision	Recall	F1
Zero-Shot	Gemini-2.0-flash	44.96%	85.92%	59.03%
	GPT-4o	59.25%	90.67%	71.66%
Few-Shot	Gemini-2.0-flash	55.30%	86.94%	67.60%
	GPT-4o	70.47%	92.86%	80.13%

Takeaway

GPT-4o with few-shot examples hits 80.13% F1, which is solid, but precision is a weak spot.

RQ3.1: BERT-like Language Models

RQ3.1: How well do smaller BERT-like models detect secrets in issue reports?

Smaller Models Do Great After Fine-Tuning

Fine-tuned BERT-family models beat proprietary LLMs by a big margin.

Table: Top BERT-like Model Results

Model	Precision	Recall	F1
CodeBERT-base	92.49%	92.91%	92.70%
LUKE-base	92.44%	92.23%	92.33%
RoBERTa-base	93.03%	91.67%	92.34%
XLNet-base-cased	88.92%	94.93%	91.83%

Key Insight

CodeBERT-base gets the best F1 (92.70%), beating GPT-4o's 80.13% by over 12 points.

RQ3.2: Fine-Tuned Open-Source LLMs

RQ3.2: How well do fine-tuned open-source LLMs detect secrets in issue reports?

Best Results Come from Fine-Tuned LLMs

Fine-tuning with QLoRA on our domain-specific data gives the best overall numbers.

Table: Fine-Tuned LLM Performance

Model	Precision	Recall	F1
LLaMA-3.1-8B	95.89%	92.00%	93.91%
Qwen-7B	94.78%	94.14%	94.46%
Gemma-7B	93.24%	94.33%	93.78%
DeepSeek-7B	92.55%	93.45%	93.00%
Mistral-7B	94.28%	90.88%	92.55%

Key Result

Qwen-7B leads with the best F1 of **94.46%**, combining the highest recall with strong precision.

A Closer Look: Qwen-7B Confusion Matrix

Well-Balanced Detection

- **7,189 / 7,235** non-secrets correctly rejected
- Only **46** false positives
- **836 / 888** secrets correctly caught
- Only **52** false negatives

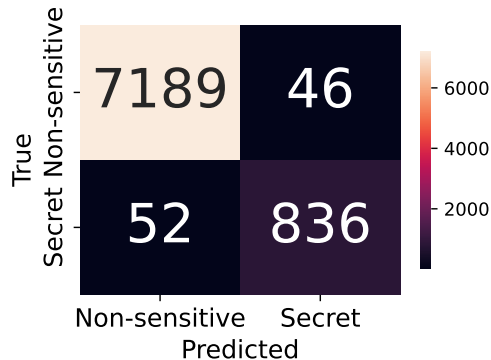


Figure: Qwen-7B confusion matrix.

How Everything Compares

Table: Summary: Regex, ML, DL Baselines vs. Transformers vs. LLMs

Method	Precision	Recall	F1
Regex	6.8%	100.0%	12.8%
Regex + Entropy	11.2%	97.5%	20.1%
Regex + Criteria Heuristic	25.39%	72.86%	37.66%
Random Forest	74.41%	46.51%	57.24%
TextCNN	93.66%	76.46%	84.19%
BERT-like (CodeBERT)	92.49%	92.91%	92.70%
Fine-tuned LLM (Qwen)	94.78%	94.14%	94.46%

Bottom Line

Fine-tuned LLMs win clearly. Regex-only methods have very low precision on issue reports.

What the Model Gets Right

Correctly Caught (True Positive):

```
"POST /v1/charges with header  
Authorization: Bearer  
sk_test_4eC39HqLyjWDarjtT1zdp7dc"
```

→ A real Stripe API key inside an API trace.

Correctly Skipped (True Negative):

```
"apiKey=xxxxxxxxxx and  
token=your_dummy_key"
```

→ Fake placeholders used for bug reproduction.

What the Model Gets Right

Correctly Caught (True Positive):

```
"POST /v1/charges with header  
Authorization: Bearer  
sk_test_4eC39HqLyjWDarjtT1zdp7dc"
```

→ A real Stripe API key inside an API trace.

Correctly Skipped (True Negative):

```
"apiKey=xxxxxxxxxx and  
token=your_dummy_key"
```

→ Fake placeholders used for bug reproduction.

Context Makes the Difference

Unlike regex tools, the model reads the **surrounding text** to tell real secrets apart from placeholders.

Why Does the Model Miss Some Secrets?

Three Reasons for Missed Detections

- 1 **Tricky placeholders:** The label says "secret" but the value is a platform name (e.g., `AIRFLOW_PASSWORD=airflow`). Human labelers and the model may disagree here.

Why Does the Model Miss Some Secrets?

Three Reasons for Missed Detections

- ❶ **Tricky placeholders:** The label says "secret" but the value is a platform name (e.g., `AIRFLOW_PASSWORD=airflow`). Human labelers and the model may disagree here.
- ❷ **Chopped-off keys:** Long RSA/crypto keys with masked parts (`****, ...`) sometimes fell outside the 200-character context window.

Why Does the Model Miss Some Secrets?

Three Reasons for Missed Detections

- ❶ **Tricky placeholders:** The label says "secret" but the value is a platform name (e.g., `AIRFLOW_PASSWORD=airflow`). Human labelers and the model may disagree here.
- ❷ **Chopped-off keys:** Long RSA/crypto keys with masked parts (`****, ...`) sometimes fell outside the 200-character context window.
- ❸ **LLM quirks:** A small number of cases where the model just gets it wrong, even with enough context.

Ablation Study: Learning Rate & Context Window

Effect of Learning Rate

Model	LR	F1
CodeBERT	1×10^{-5}	91.85%
CodeBERT	2×10^{-5}	92.70%
CodeBERT	4×10^{-5}	91.10%
Qwen	1×10^{-4}	93.70%
Qwen	2×10^{-4}	94.46%
Qwen	4×10^{-4}	93.17%

Effect of Context Window Size

Model	Chars	F1
CodeBERT	100	91.95%
CodeBERT	200	92.70%
CodeBERT	300	92.79%
Qwen	100	93.90%
Qwen	200	94.46%
Qwen	300	94.55%

What We Found

200 characters is the sweet spot. Going beyond 300 characters barely helps, which shows our pipeline is already efficient.

Real-World Test: 178 GitHub Repositories

Does It Work in the Wild?

We tested on **178 real-world repos** (terraform-provider-aws, google-api-nodejs-client, vercel, redis, ...) with **1,489 issue reports** and only 30 actual secrets.

Table: Real-World Evaluation Results

Class	Precision	Recall	F1
Secret	50.98%	86.67%	64.20%
Macro Average	75.35%	92.45%	81.60%

What This Means

With 86.67% recall, the model catches most real secrets. It works well beyond our curated dataset.

Experimental Setup

- **Hardware:** Intel Core i5-13400F, 128 GB RAM, NVIDIA RTX 4090
- **QLoRA + PEFT:** Fine-tuning only needs **8 to 10 GB VRAM** for 7B to 8B models
- **Inference VRAM:** Under **6 GB** for all models
- **Inference Speed:** **0.05 to 0.13 seconds** per candidate

Table: Resource Usage

Family	Train VRAM	Train Time	Infer VRAM	Infer Time
BERT-like	6 to 7 GB	2 to 3 hrs	3 to 4 GB	0.05 to 0.07s
LLMs	8 to 10 GB	14 to 16 hrs	5 to 6 GB	0.10 to 0.13s

Conclusion & Future Work

What We Found

- This is the **first large-scale study** of secret leaks in software issue reports.
- Our hybrid pipeline (regex + LLM) clearly beats traditional methods.
- Fine-tuned **Qwen-7B** reaches **94.46% F1**, and **CodeBERT** hits **92.70%**.
- Tested on **178 real-world repos** with **81.6% F1**.
- Everything runs on **regular consumer hardware**.

What is Next

- Add retrieval-based methods to help the model reason better.
- Try better prompting strategies for improved context understanding.
- Grow the dataset with more diverse and realistic issue reports.

References I

-  Runhan Feng, Ziyang Yan, Shiyan Peng, and Yuanyuan Zhang.
Automated detection of password leakage from public github repositories.
In *Proceedings of the 44th International Conference on Software Engineering*, pages 175–186, 2022.
-  GitGuardian.
ggshield.
<https://www.gitguardian.com/ggshield>, 2017.
Accessed: 2024-02-02.
-  GitGuardian.
State of secrets sprawl report 2023, 2024.

References II



GitGuardian.

The state of secrets sprawl report 2024.

<https://www.gitguardian.com/state-of-secrets-sprawl-report-2024>, 2024.

Accessed: 2025-04-16.



Gitleaks.

Gitleaks.

<https://github.com/gitleaks/gitleaks>, 2018.

Accessed: 2024-02-02.



Michael Meli, Matthew R McNiece, and Bradley Reaves.

How bad can it git? characterizing secret leakage in public github repositories.

In *NDSS*, 2019.

References III

-  Md Nafiu Rahman, Sadif Ahmed, Zahin Wahab, S M Sohan, and Rifat Shahriyar. Secret breach detection in source code with large language models. *arXiv preprint arXiv:2504.18784*, 2025.
-  Aakanksha Saha, Tamara Denning, Vivek Srikumar, and Sneha Kumar Kasera. Secrets in source code: Reducing false positives using machine learning. In *2020 International Conference on COMmunication Systems & NETworkS (COMSNETS)*, pages 168–175. IEEE, 2020.
-  Vibha Singhal Sinha, Diptikalyan Saha, Pankaj Dhoolia, Rohan Padhye, and Senthil Mani. Detecting and mitigating secret-key leaks in source code repositories. In *2015 IEEE/ACM 12th Working Conference on Mining Software Repositories*, pages 396–400. IEEE, 2015.

References IV



Sophos.

Sophos 2023 threat report.

<https://www.sophos.com/en-us/content/security-threat-report>, 2023.

Accessed: 2024-03-21.



TruffleSecurity.

Trufflehog.

<https://github.com/trufflesecurity/trufflehog>, 2016.

Accessed: 2024-02-02.

Thank You

Replication Package

Our code and data are available here [link]

Contact or Questions?

nafiu.rahman@gmail.com